arXiv (2021) Submitted December/21; Published December/21

arXiv:2112.06560v1 [cs.LG] 13 Dec 2021

HiClass: a Python library for local hierarchical classification compatible with scikit-learn

Fábio M. Miranda* FABIO.MALCHERMIRANDA@HPI.DE
Hasso Plattner Institute, Digital Engineering Faculty, University of Potsdam, 14482 Potsdam, Germany

Niklas Köehnecke* NIKLAS.KOEHNECKE@STUDENT.HPI.UNI-POTSDAM.DE
Hasso Plattner Institute, Digital Engineering Faculty, University of Potsdam, 14482 Potsdam, Germany

Bernhard Y. Renard BERNHARD.RENARD@HPI.DE
Hasso Plattner Institute, Digital Engineering Faculty, University of Potsdam, 14482 Potsdam, Germany

Editor: Unknown

Abstract

`HiClass` is an open-source Python package for local hierarchical classification fully compatible with `scikit-learn`. It provides implementations of the most popular machine learning models for local hierarchical classification, including Local Classifier Per Node, Local Classifier Per Parent Node and Local Classifier Per Level. In addition, the library includes tools to evaluate model performance on hierarchical data. The documentation contains installation instructions, interactive notebooks, and a complete description of the API. `HiClass` is distributed under the simplified BSD license, encouraging its use in both academic and commercial settings. Source code and documentation are available at `https://gitlab.com/dacs-hpi/hiclass`.

Keywords: Local Hierarchical Classification, Supervised Learning, Python

1. Introduction

Many classification problems across different application domains can be naturally modeled hierarchically (Figure 1), typically in the form of trees or directed acyclic graphs (Silla and Freitas, 2011). Examples of hierarchical classification problems are vast, ranging from musical genre classification (Ariyaratne and Zhang, 2012; Iloga et al., 2018) to text categorization (Javed et al., 2021; Ma et al., 2022), taxonomic classification of viral sequences in metagenomic data (Shang and Sun, 2021) and identification of COVID-19 in chest X-ray images (Pereira et al., 2020).

Nonetheless, many classifiers proposed in the literature are designed to completely ignore the existing hierarchy between classes, usually predicting only leaf nodes, in a methodology known as flat approach. Although easy to implement, the flat approach is incapable of dealing with problems where prediction of leaf nodes is not mandatory. Furthermore, since they consider the hierarchy during training, hierarchical models generally produce better results when compared with flat approaches, consistently achieving improvements in predic-

*Fábio M. Miranda and Niklas Koehnecke contributed equally to this article.

©2021 Fábio M. Miranda, Niklas Koehnecke and Bernhard Y. Renard.

tive performance (Silla and Freitas, 2011). Hence, in this manuscript we introduce HiClass, a python library that implements the most common patterns for local hierarchical classifiers (Figure 1), which can be employed in different application domains where the data can be hierarchically structured in the shape of trees or directed acyclic graphs. While ideally the full hierarchical structure is known, classification is also possible with only partially known structures and missing values in intermediate levels of the hierarchy.

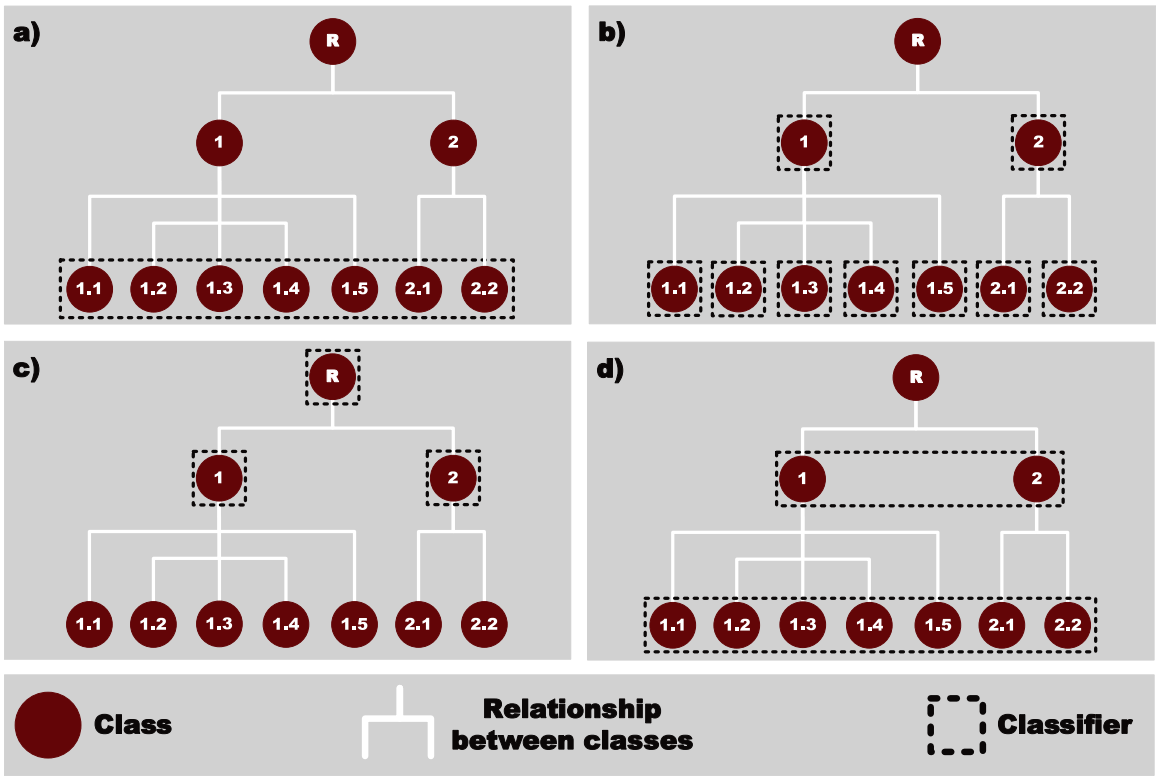

Figure 1: Depiction of the flat approach and the local hierarchical classification models implemented in HiClass, for a two-level hierarchy. a) Flat approach, where a single multi-class classifier is trained to predict only the leaf nodes in hierarchical data, completely ignoring previous levels of the hierarchy. b) Local classifier per node, where several binary classifiers are trained for each existing node in the hierarchy, excluding the root node. c) Local classifier per parent node, where various multi-class classifiers are trained for each parent node in the hierarchy, with the goal of predicting the children nodes. d) Local classifier per level, where a multi-class classifier is trained for each level in the hierarchy.

2. Overview and Design

The API of `HiClass` is designed to be compatible with the `scikit-learn` API (Pedregosa et al., 2011), offering users a familiar API to train hierarchical models via `fit()`, store

trained models with `pickle` and predict labels with the `predict()` method. Moreover, each local classifier in `HiClass` is by default scikit-learn's `BaseEstimator` class, enabling users to employ any of the classifiers already implemented in scikit-learn's library. However, classes other than the BaseEstimator can also be used, as long as they implement the methods `fit()`, `predict()` and `clone()`, thus enabling users to code their own local classifiers using third-party libraries, e.g., `TensorFlow` (Abadi et al., 2016) or `PyTorch` (Paszke et al., 2019).

To allow a better integration with `scikit-learn` and faster training, `HiClass` uses for both features and labels numpy's structured arrays, whose data type is a composition of simpler data types (Harris et al., 2020). While features are exactly the same shape as expected for training flat models in scikit-learn, hierarchical training labels are represented as an array of shape `n_samples` $\times$ `n_levels`, where each column must contain either a label for the respective level in the hierarchy or a placeholder that indicates missing labels in the hierarchical structure.

The directed acyclic graph (DAG) implemented in the library `NetworkX` (Hagberg et al., 2008) was the data structure chosen to make the models as generic as possible, since hierarchical data in the form of trees can also be represented with a DAG. Training is performed in parallel for each local classifier, while prediction, which is not a bottleneck, is performed from top to bottom, following the hierarchical structure to keep consistency among the several hierarchical levels.

According to Silla and Freitas (2011), the use of flat classification metrics might not be adequate to give enough insight of which algorithm is better at classifying hierarchical data. Hence, in `HiClass` we implemented the metrics of hierarchical precision (hP), hierarchical recall (hR) and hierarchical f-score (hF), which are extensions of the renowned metrics of precision, recall and f-score, but tailored to the hierarchical classification scenario. These hierarchical counterparts were initially proposed by Kiritchenko et al. (2006), and are defined as follows:

$$hP = \frac{\sum_i |\alpha_i \cap \beta_i|}{\sum_i |\alpha_i|}, \; hR = \frac{\sum_i |\alpha_i \cap \beta_i|}{\sum_i |\beta_i|}, \; hF = \frac{2 \times hP \times hR}{hP + hR}$$

where α_i is the set consisting of the most specific classes predicted for test example i and all their ancestor classes, while β_i is the set containing the true most specific classes of test example i and all their ancestors, with summations computed over all test examples.

3. Code Quality Assurance

To ensure high code quality, `inheritance` is applied whenever possible to keep the code concise and easier to maintain, and all implementations adhere to the `PEP 8` code style (Van Rossum et al., 2001) enforced by `flake8` and the uncompromising code formatter `black`. Documentation is provided through `docstrings` (Goodger and van Rossum, 2010), and the implementations are accompanied by `unit tests` that cover `99%` of our code and are automatically executed by our `continuous integration workflow` upon commits.

4. Installation and Usage

HiClass is hosted on GitLab[1], interactive tutorials in the form of notebooks are available on the platform Google Colab[2] and API documentation is hosted on Read the Docs[3]. Packages for Python 3.8.3 and above are available for Linux, macOS and Windows and can be obtained with `pip install hiclass` or `conda install -c conda-forge hiclass`. Algorithm 1 shows a basic example of fitting and evaluating a local hierarchical model with `HiClass`. More elaborate examples can be found in the interactive tutorials.

```python
from hiclass.metrics import f1
from hiclass import LocalClassifierPerNode
from sklearn.ensemble import RandomForestClassifier

# Import data
X_train, X_test = get_features()  # (n_samples, n_features)
Y_train, Y_test = get_labels()  # (n_samples, n_levels)

# Use random forest classifiers for every node
rf = RandomForestClassifier()
lcpn = LocalClassifierPerNode(local_classifier=rf)
lcpn.fit(X_train, Y_train)  # Train hierarchical classifier
predictions = lcpn.predict(X_test)  # Predict test data

# Print hierarchical f-score
print("f1: %.2f" % (round(f1(y_true=Y_test, y_pred=predictions), 3)))
```

Algorithm 1: Example on how to use HiClass to train and evaluate a hierarchical classifier.

5. Comparison with Flat Classifiers

While `HiClass` focuses on ease of use and is fully written in a high level language, care has been taken to maximize computational efficiency. In Figure 2, we compare the hierarchical f-score and computational resources (measured with the command `time`) between a couple of flat classifiers with default parameters from the library scikit-learn and the local hierarchical classifiers implemented in `HiClass`, where each local classifier contains the same parameters of the corresponding flat classifier (see Supplementary Table S1 for hierarchical precision and recall). For comparison purposes, we used the amazon product review dataset[4], which contains 40,000 instances for training and 10,000 more for validation. The benchmark was computed on a cluster node running GNU/Linux with 512 GB physical memory and 128 cores provided by two AMD EPYC™ 7742 processors.

Our results reveal that the hierarchical f-score was enhanced by all local hierarchical approaches when compared with the logistic regression flat classifier, while training time and memory usage were greatly reduced with the local classifiers per node and per parent node. However, when compared with the random forest flat classifier, only the local classifier

1. https://gitlab.com/dacs-hpi/hiclass
2. https://colab.research.google.com/drive/1Idzht9dNoB85pjc9gOL24t9ksrXZEA-9?usp=sharing
3. https://hiclass.readthedocs.io/
4. https://www.kaggle.com/kashnitsky/hierarchical-text-classification

per node improved the hierarchical f-score, but both local classifiers per node and per parent node still showed a considerable decline in memory consumption.

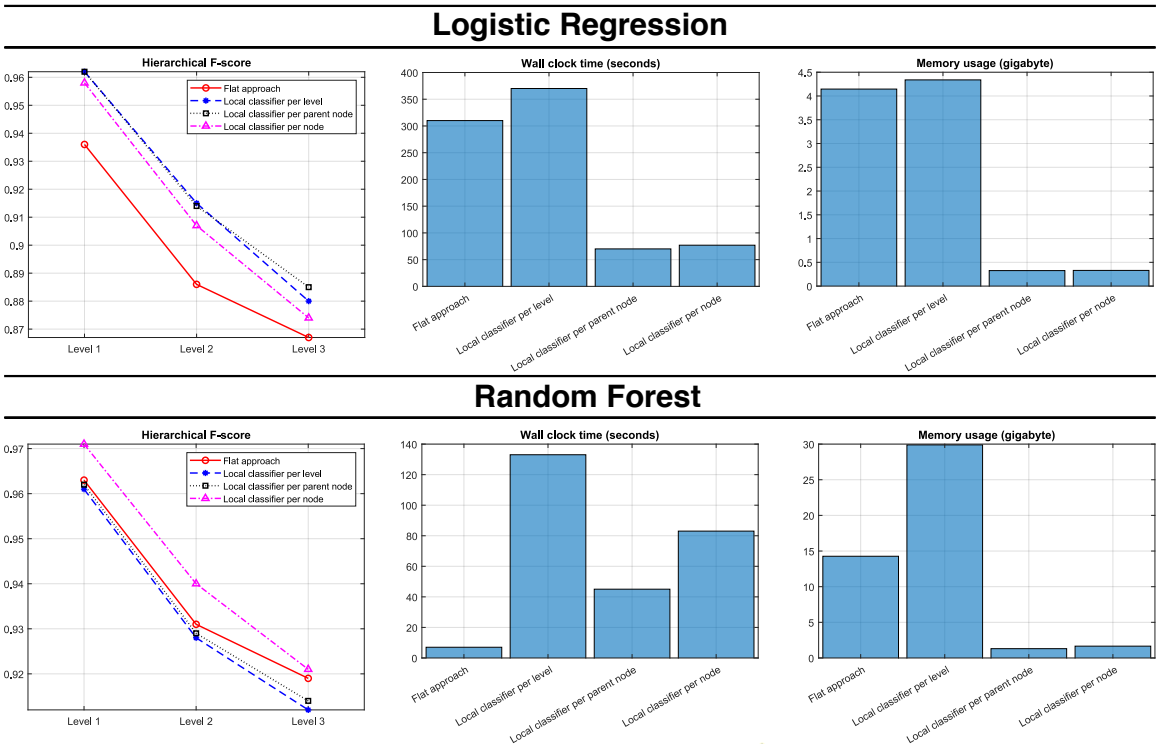

Figure 2: Comparison between flat approaches and local hierarchical classifiers implemented in `HiClass`. For this benchmark, the metrics used were hierarchical f-score computed for the 3 different hierarchical levels in the amazon product review dataset, time in seconds and memory usage in gigabyte. Overall, the hierarchical classifier per node improved the hierarchical f-score and reduced memory consumption when compared with the flat approaches.

6. Conclusion

HiClass is a Python package that provides implementations of popular machine learning models and evaluation metrics for local hierarchical classification. Thanks to its compatibility with the scikit-learn API, users can choose among existing classifiers to create powerful hierarchical classifiers for hierarchical data.

Acknowledgments

BYR gratefully acknowledges support by the BMBF-funded Computational Life Science initiative (project DeepPath, 031L0208, to B.Y.R.).

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*, pages 265–283, 2016.

Hasitha B Ariyaratne and Dengsheng Zhang. A novel automatic hierachical approach to music genre classification. In *2012 IEEE International Conference on Multimedia and Expo Workshops*, pages 564–569. IEEE, 2012.

David Goodger and Guido van Rossum. Docstring conventions. In *Pro Python*, pages 303–307. Springer, 2010.

Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

Sylvain Iloga, Olivier Romain, and Maurice Tchuenté. A sequential pattern mining approach to design taxonomies for hierarchical music genre recognition. *Pattern Analysis and Applications*, 21(2):363–380, 2018.

Taimoor Ahmed Javed, Waseem Shahzad, and Umair Arshad. Hierarchical text classification of urdu news using deep neural network. *arXiv preprint arXiv:2107.03141*, 2021.

Svetlana Kiritchenko, Stan Matwin, Richard Nock, and A Fazel Famili. Learning and evaluation in the presence of class hierarchies: Application to text categorization. In *Conference of the Canadian Society for Computational Studies of Intelligence*, pages 395–406. Springer, 2006.

Yinglong Ma, Xiaofeng Liu, Lijiao Zhao, Yue Liang, Peng Zhang, and Beihong Jin. Hybrid embedding-based text representation for hierarchical multi-label text classification. *Expert Systems with Applications*, 187:115905, 2022.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32:8026–8037, 2019.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.

Rodolfo M Pereira, Diego Bertolini, Lucas O Teixeira, Carlos N Silla Jr, and Yandre MG Costa. Covid-19 identification in chest x-ray images on flat and hierarchical classification scenarios. *Computer Methods and Programs in Biomedicine*, 194:105532, 2020.

Jiayu Shang and Yanni Sun. Cheer: hierarchical taxonomic classification for viral metagenomic data via deep learning. *Methods*, 189:95–103, 2021.

Carlos N Silla and Alex A Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1):31–72, 2011.

Guido Van Rossum, Barry Warsaw, and Nick Coghlan. Pep 8: style guide for python code. *Python. org*, 1565, 2001.